

PanRocq

Florent Schaffhauser

2026.09.16

Table of contents

Introduction	3
1 Setup	4
1.1 Quarto	4
1.2 Typst	5
2 Usage	6
2.1 Quarto	6
2.1.1 Rendering	6
2.1.2 Source	6
2.2 Typst	6
2.2.1 Rendering	6
2.2.2 Source	7
3 Troubleshooting	8
3.1 Quarto	8
3.2 Typst	8
3.3 Unicode	9
3.4 Code blocks background	9

Introduction

This is an example of a [Quarto book](#) created using PanRocq in order to format and highlight Rocq code in the html and pdf renderings of the book. PanRocq also works with [Typst](#).

PanRocq's formatting and highlighting rules are based on the [Rocq-sublime](#) syntax by Matthieu Sozeau (MIT Licence, 2025).

1 Setup

1.1 Quarto

Add the files `PanRocq.theme` and `PanRocq.xml` to the root folder of your Quarto project. Then add the following `syntax-definitions` and `highlight-style` fields to the `_quarto.yml` file.

```
format:
  html:
    theme: cosmo
    syntax-definitions:
      - ./PanRocq.xml
    highlight-style: ./PanRocq.theme
  pdf:
    documentclass: scrreprt
    syntax-definitions:
      - ./PanRocq.xml
    highlight-style: ./PanRocq.theme
```

Any Quarto code block starting with a `rocq` fence will then be highlighted. See [Section 2.1](#) for an example. `PanRocq` is intended as a highlighting tool in light mode, where it matches standard `Rocq` highlighting rules. However, we have also included a version for Quarto's dark mode (see `PanRocq-dark.theme`), which can be configured as follows.

```
format:
  html:
    theme:
      light: [cosmo, brand]
      dark: [solar, brand]
    css: styles.css
    syntax-definitions:
      - ./PanRocq.xml
    highlight-style:
      light: ./PanRocq.theme
      dark: ./PanRocq-dark.theme
```

It renders reasonably well with the [Quarto themes](#) darkly, slate, solar and superhero, but in that case we recommend importing the `styles.css` file that we provide if you want to use a scroll bar to deal with code block overflows.

1.2 Typst

Add the files `Rocq.sublime-syntax` and `Rocq.tmTheme` to your Typst directory. Then add the following to your `.typ` file.

```
#set raw(syntaxes: "Rocq.sublime-syntax", lang: "rocq", theme: "Rocq.tmTheme")
```

Any Typst code block starting with a `rocq` fence will then be highlighted. See [Section 2.2](#) for an example.

2 Usage

2.1 Quarto

2.1.1 Rendering

```
Definition negb (b : bool) : bool :=  
  match b with  
  | true  => false  
  | false => true  
end.
```

2.1.2 Source

```
```rocq  
Definition negb (b : bool) : bool :=
 match b with
 | true => false
 | false => true
end.
```
```

2.2 Typst

2.2.1 Rendering

When you compile your `.typ` file, the code blocks will be rendered using PanRocq in the `.pdf` file.

```
typst c Typst/Exercises.typ
```

2.2.2 Source

```
```rocq
Inductive list (A : Type) : Type :=
| nil : list A
| cons : A → list A → list A.
```
```

3 Troubleshooting

3.1 Quarto

This book was compiled with Quarto 1.9.37 using the lua-latex engine. The LaTeX package `fvextra` is used to wrap lines in code blocks when rendering .qmd to .pdf. This is set up as follows in the `_quarto.yml` file.

```
pdf:
  documentclass: scrreprt
  latex-tinytex: false
  pdf-engine: lualatex
  code-overflow: wrap
  include-in-header:
    text: |
      \usepackage{fvextra}
      \DefineVerbatimEnvironment{Highlighting}{Verbatim}{
        breaklines, commandchars = \\\{\}
      } % this is used to wrap lines in code blocks when rendering .qmd to
  ↪ .pdf
  syntax-definitions:
    - ./PanRocq.xml
  highlight-style: ./PanRocq.theme
```

3.2 Typst

Occasionally, you might need to add a missing Unicode character to `Rocq.sublime-syntax`, for instance U+227C to render $\preccurlyeq$, which is `\preccurlyeq` in LaTeX ($\preccurlyeq$), and which can be added to the `operator` field of `Rocq.sublime-syntax`.

```
variables:
  operator: '(<existing_syntax>|\u{227C})'
```

3.3 Unicode

Additional support is often needed to render Unicode characters properly in the pdf file. This is set up as follows in the `_quarto.yml` file. Note that a character such as $\leq$ should still be put behind backticks to render properly in our setup ($\leq$ is obtained by typing ``≤`` in the source file). Change Latin Modern Roman to DejaVu Sans or similar to be able to use such Unicode characters without backticks.

```
pdf:
  documentclass: scrreprt
  latex-tinytex: false
  pdf-engine: lualatex
  include-in-header:
    text: |
      \directlua{luaotfload.add_fallback("unicodedefallbackinprose",
↪ {"NotoEmoji:mode=harf;"})}
      \directlua{luaotfload.add_fallback("unicodedefallbackincodeblocks",
↪ {"NotoEmoji:mode=harf;", "STIX Two Math:mode=harf;"})}
      \setmainfont{Latin Modern
↪ Roman}[RawFeature={fallback=unicodedefallbackinprose}]
      \setmonofont{DejaVu Sans
↪ Mono}[RawFeature={fallback=unicodedefallbackincodeblocks}]
```

To have `:smile:` render as ☺ in the HTML version, add the following to `_quarto.yml`.

```
from: markdown+emoji
```

These will render differently in the pdf file, even if the emoji itself is copy-pasted and put behind backticks ☺ .

3.4 Code blocks background

For convenience, the `"background-color"` field in the `PanRocq.theme` file has been set to `null`. It can be customized, by replacing `null` by `"#eef2f6"` or any other color. Alternatively, one can set `code-block-bg: "#eef2f6"` in `_quarto.yml` (but simply setting `code-block-bg: true` does not suffice to override `null` in the pdf version of the book).

For more on Quarto's code highlighting features, visit <https://quarto.org/docs/output-formats/html-code.html#highlighting>.